「エクストリームプログラミング」から読み解く現代ソフトウェア開発

ストーリーポイントと自動テストの功罪

2020/6/26 Scrum Fest Osaka 2020

大中浩行(@setoazusa)

この作品は クリエイティブ・コモンズ 表示 4.0 国際 ライセンスの下に提供されています。

Copyright 2020 Hiroyuki Onaka

こんにちは！

- 大中浩行(@setoazusa / せとあず)
- XP祭り実行委員会

Copyright 2020 Hiroyuki Onaka

私の立ち位置

- アプリケーションエンジニアかつマネージャー
- SIerかつツールの代理店の中の人
- 領域的には「エンタープライズアジャイル」というもの

Copyright 2020 Hiroyuki Onaka

本発表では、2018年のソフトウェア開発の現場のようすと、『エクストリームプログラミング』（通称「XP白本」）が描いたエクストリームプログラミングの世界を重ねながら、XPというものが描こうとしたエンジニアリングの世界を今日の世界に描き出すことを目標とします。

Copyright 2020 Hiroyuki Onaka

今日のテーマ

XPのプラクティスの中でもよく使われそして誤解される「ストーリー（ストーリーポイント）」と「自動テスト（テストファーストプログラミング）」について今日はお話しします。

Copyright 2020 Hiroyuki Onaka

XPとは

Copyright 2020 Hiroyuki Onaka

- Extreme Programming Explained: Embrace Change (2nd Edition)
  (角征典訳「エクストリームプログラミング」)
- 通称: XP白本

Copyright 2020 Hiroyuki Onaka

XPとは

「エクストリームプログラミング（XP）はソーシャルチェンジである。」

Copyright 2020 Hiroyuki Onaka

ソーシャルチェンジとは

…？

Copyright 2020 Hiroyuki Onaka

訳者(角さん)による解説

- XP は必ずしも「社会」を変えるための活動ではない。隣の席にいるプログラマーとうまくコミュニケーションしたり、顧客と密接にやりとりをしたり、ユーザーが安心して使えるソフトウェアを届けたりする。そうした「人間関係」を扱う活動こそが、エクストリームプログラミングである。

Copyright 2020 Hiroyuki Onaka

「ソーシャル」とは

「ソーシャルディスタンス」と同じコンテキストですね

Copyright 2020 Hiroyuki Onaka

XPが重視する「価値」

- コミュニケーション
- シンプリティ
- フィードバック
- 勇気
- リスペクト

Copyright 2020 Hiroyuki Onaka

価値、原則、プラクティス

「ポールは造園の善しあしを判断する高度な感覚を持っている。彼は庭全体を見て、何がうまくいっていて、何がうまくいっていないかを直感的に把握できる。枝刈りがうまくできて私が得意気になっているとしたら、ポールは木全体の生え具合を見ているのである。それは、彼のほうが私よりも枝刈りがうまいからではない。庭に作用するフォースを彼が感じ取っているからである。彼にとっては簡単で明白であっても、私は懸命に取り組まなければいけない。」

Copyright 2020 Hiroyuki Onaka

価値とは(2)

「このレベルの知識や理解を価値（values）と呼ぼう。」

「ソフトウェア開発にかかわる人であれば、何が重要であるかの感覚を持っている。考えられるすべての設計を実装前に注意深く考えることが重要だと考える人もいれば、個人の自由が制限されないことが重要だと考える人もいる。」

Copyright 2020 Hiroyuki Onaka

「価値」

「価値というのは、とても抽象的な表現なので、価値という名の下にどんなことでもできてしまう。」

こういうオールマイティーに使える言葉って使い方難しいですよね

Copyright 2020 Hiroyuki Onaka

やっぱ、こういうのを実践したらXPなんですかね

- 全員同席
- チーム全体
- 情報満載のワークスペース
- いききとした仕事
- ペアプログラミング
- ストーリー
- 週次サイクル
- 四半期サイクル
- ゆとり
- 10分ビルド
- 継続的インテグレーション
- テストファーストプログラミング
- インクリメンタルの設計

Copyright 2020 Hiroyuki Onaka

何をやったら(アジャイル|スクラム|XP)なのか

「アジャイルって言うけど本当にオンサイト顧客とかやらないでしょ？」

「業務でテストファーストなんかやらないよ」

Copyright 2020 Hiroyuki Onaka

今日のテーマ

XPのプラクティスの中でもよく使われそして誤解される「ストーリー（ストーリーポイント）」と「自動テスト（テストファーストプログラミング）」について今日はお話しします。

Copyright 2020 Hiroyuki Onaka

ストーリーポイント

Copyright 2020 Hiroyuki Onaka

ストーリーポイントの起源

ストーリーポイントの発案者は、アジャイルマニフェストの発起人であり、世界で初めてのエクストリームプログラミング(XP)のチームにコーチとして関わった、Ron Jeffries氏とされています。

Copyright 2020 Hiroyuki Onaka

「XPエクストリーム・プログラミング導入編」から

さて、どうしたらいいのか教えてあげよう。私たちは、簡単なポイント制を使ってストーリーの難易度を見積もる。このポイントの呼び方はチームのネーミングルールに従ってもらって構わない。このポイントを完全作業週(perfect engineering week)と呼ぶチームもある。ただし、この呼び名を使うのがよくないと考えるチームもある。完全週数を実際の週数(結局はこれが私たちが使える時間だ)にあわせようとして、結局自分たちにプレッシャーをかけすぎることになるというのが理由だ。また「グミペアー」と呼んでいるチームもある。いや、本当に。あなたは「ストーリー見積単位」とか、「ドル」と呼ぶほうがいい、と思うかもしれない。Ronの今週のお気に入りは、ただの「ポイント」という呼び名だ。

Copyright 2020 Hiroyuki Onaka

https://twitter.com/RonJeffries/status/307469737305186304?ref_src=twsrc%5Etfw

私がストーリーポイントを発明したか、その場所に私がいたのです。ストーリーポイントとは単に**時間の湾曲表現**であり、それ以上でも以下でもありません。(発表者訳)

Copyright 2020 Hiroyuki Onaka

もともとのストーリーポイントの考えは、工数の全てを作業に用いることが出来る時間と、実際に作業時間の差に対応するために、工数をポイントとして扱う、というものでした。

Copyright 2020 Hiroyuki Onaka

ストーリーポイントのひろがり

- Mike Cohn氏(「アジャイルな見積もりと計画づくり」の著者)
- 「User Stories Applied: For Agile Software Development」の中でストーリーポイントを取り上げるなどする
- XPに限らずアジャイル開発全般にストーリーポイントというものが広まっていきます。

Copyright 2020 Hiroyuki Onaka

もっともその後XPではストーリーポイントによる見積もりはプラクティスから外されました。

Copyright 2020 Hiroyuki Onaka

「エクストリームプログラミング」から

第1版では、もっと抽象的な見積りモデルを紹介した。ストーリーのコストに1、2、3 という「ポイント」をつけていたのである。大きなストーリーは計画する前に分割する必要があった。ストーリーの実装が始まったときに、1 週間で達成できるポイント数をすばやく把握するのである。だが、今では**実時間で見積もるほうが私の好みだ。そのほうができるだけ明確で、直接的で、透明性のあるコミュニケーションがとれるからである。**

Copyright 2020 Hiroyuki Onaka

スクラムでは？

スクラムは見積の単位に対して言及することを巧みに回避しており、「スクラムガイド」の中にはストーリーポイントについての言及はありません。

Copyright 2020 Hiroyuki Onaka

色々なコンテキストが付け加わる

- 相対見積もり
- 過剰バッファーへの対処
- コミットメントと進捗のトラッキングの分離
- チームの生産性の可視化

Copyright 2020 Hiroyuki Onaka

「アジャイルサムライ」から

「研究調査によると、人間は相対見積りならうまくこなせるらしい。たとえば、目の前に岩が転がっていたとしよう。このとき、片方の岩がもう片方の岩に対してどれぐらい大きいかは、非常に正確に見積もれるそうだ。」

Copyright 2020 Hiroyuki Onaka

過剰バッファーへの対処

- 個別に工数を見積もると、個別のバッファーが積み重なっていって過剰なバッファーを生み出すことがあります。
- 過剰バッファーを避けるために、個別のストーリーの規模はポイントで見積もり、バッファーは全体で確保しましょう

Copyright 2020 Hiroyuki Onaka

過剰バッファーへの対処

- これ見積もり手法の話であってストーリーポイントに限った話じゃねえな…

Copyright 2020 Hiroyuki Onaka

コミットメントと進捗のトラッキングの分離

- 過剰にバッファーを積んでしまうかといえば、個人がタスクを見積もった工数で終わらせることにコミットメントしてしまうからです！
- ストーリーポイントで見積もることによって、見積もりにコミットメントすることをやめましょう！

Copyright 2020 Hiroyuki Onaka

コミットメントと進捗のトラッキングの分離

- チームがストーリーポイントに対してコミットメントするようになったら同じですよね…

Copyright 2020 Hiroyuki Onaka

チームの生産性の可視化

- 新しくメンバーが集まり、チームが結成されたとき、チームはステークホルダーに対しチームとして機能していることをステークホルダーに対してアピールする必要があります。
- この際に指標として有効なのはストーリーポイントの累積値としてのベロシティーです。

Copyright 2020 Hiroyuki Onaka

でも生産性の可視化って

- チームがどれだけの量アウトプットを出せたかっていう生産性ありきですよね…
- ソフトウェア開発ってそうだっけ？アジャイルマニフェストで言ってた「価値のあるソフトウェア」の話はどこにいった

Copyright 2020 Hiroyuki Onaka

ストーリーポイントには色々な問題があります

【翻訳】ストーリーポイントを使うのをやめよう - PoohSunny's blog

http://poohsunny.hatenablog.com/entry/2017/12/18/235235

Copyright 2020 Hiroyuki Onaka

とはいうものの

- 過剰バッファーを積んでしまう現場とか、
- 見積もりにコミットメントしてしまう現場とか、
- チームの生産性を可視化できない現場とか、

Copyright 2020 Hiroyuki Onaka

そういう現場をどう改善していくかということを考えると、ストーリーポイントをどう活用するかという視点に戻らざるを得ない

Copyright 2020 Hiroyuki Onaka

なぜ「プランニングポーカー」があるか

- プランニングポーカーのプラクティスは、見積もりに対してステークホルダーがコミュニケーションすることに価値を置いているからにほかならない。
- スケジュールと見積もりへのコミットメントよりも、コミュニケーションを通じたそれへの合意

Copyright 2020 Hiroyuki Onaka

「ストーリーポイント」が生きる使い方

「プロセスやツールよりも個人と対話を」

Copyright 2020 Hiroyuki Onaka

おたよりコーナー

Copyright 2020 Hiroyuki Onaka

自動テスト

Copyright 2020 Hiroyuki Onaka

自動テスト/テストファーストプログラミング/テスト駆動開発

XPでは「テストファーストプログラミング」「自動テスト」

その後「テスト駆動開発」という概念がうまれる

Copyright 2020 Hiroyuki Onaka

XPの「テストファーストプログラミング」の定義

コードを変更する前に、失敗する自動テストを書くこと。

Copyright 2020 Hiroyuki Onaka

テストファーストプログラミングのメリット

- スコープクリープ
  - プログラムのあるべき姿を定義することによって、本来のスコープに集中する
- 結合度と凝集度
  - 疎結合で凝集度の高いコードは、テストしやすい
- 信頼
  - **「動かないコードの作者を信頼するのは難しい。」**
- リズム
  - テスト→コード→リファクタ（→テスト）のリズム

Copyright 2020 Hiroyuki Onaka

自動テストにも色々なコンテキストがついてくる

- 品質保証
- リアルオプション
- テストがないコードはレガシーコードだ！

Copyright 2020 Hiroyuki Onaka

品質保証

- 自動テストが品質保証のプロセスに組み込まれることによる、テストを書くことの義務化。
- スローテストや不安定なテスト(Fragile Test)による、技術的負債(ネガティブな意味)となっているが捨てられないテスト

Copyright 2020 Hiroyuki Onaka

品質に関わるもう一つの問題

ユニットテストが自動テストの中心となることによる、「テストはしたが動かないプログラム」

Copyright 2020 Hiroyuki Onaka

「テストはしたが動かないプログラム」

- ユニットテストの中でのメソッド呼び出しと、エンドユーザーの操作時では呼び出し方が違う。
- 動かないプログラムより包括的なドキュメント

Copyright 2020 Hiroyuki Onaka

このことに取り組んだ先駆者たち

- 日本だと故石井勝(まさーる)氏
  - シナリオベースのテストケースの書き方
    - http://objectclub.jp/community/memorial/homepage3.nifty.com/masarl/article/junit/scenario-based-testcase.html
- 海外だとDan North氏
  - テストメソッドのメソッド名を、テストに対するドキュメンテーションとして使用するという着眼点からの振る舞い駆動(Behavior Driven)という考え→BDDへと発展

Copyright 2020 Hiroyuki Onaka

BDDにおける「リアルオプション」

オプション取引の考え方を事業評価に応用したもの。事業を進める上で、将来に選択する柔軟性をどれだけ持っているかを評価するもの。

BDDを巡る議論では、テストによる仕様記述が果たす役割としてこの「リアルオプション」の考えが出てくる。

Copyright 2020 Hiroyuki Onaka

ただ、テストファーストプログラミングは、プログラマーのための方法論なので、このことにより自動テストは「ビジネスにコミットするテスト」という重荷を負うことになりました。

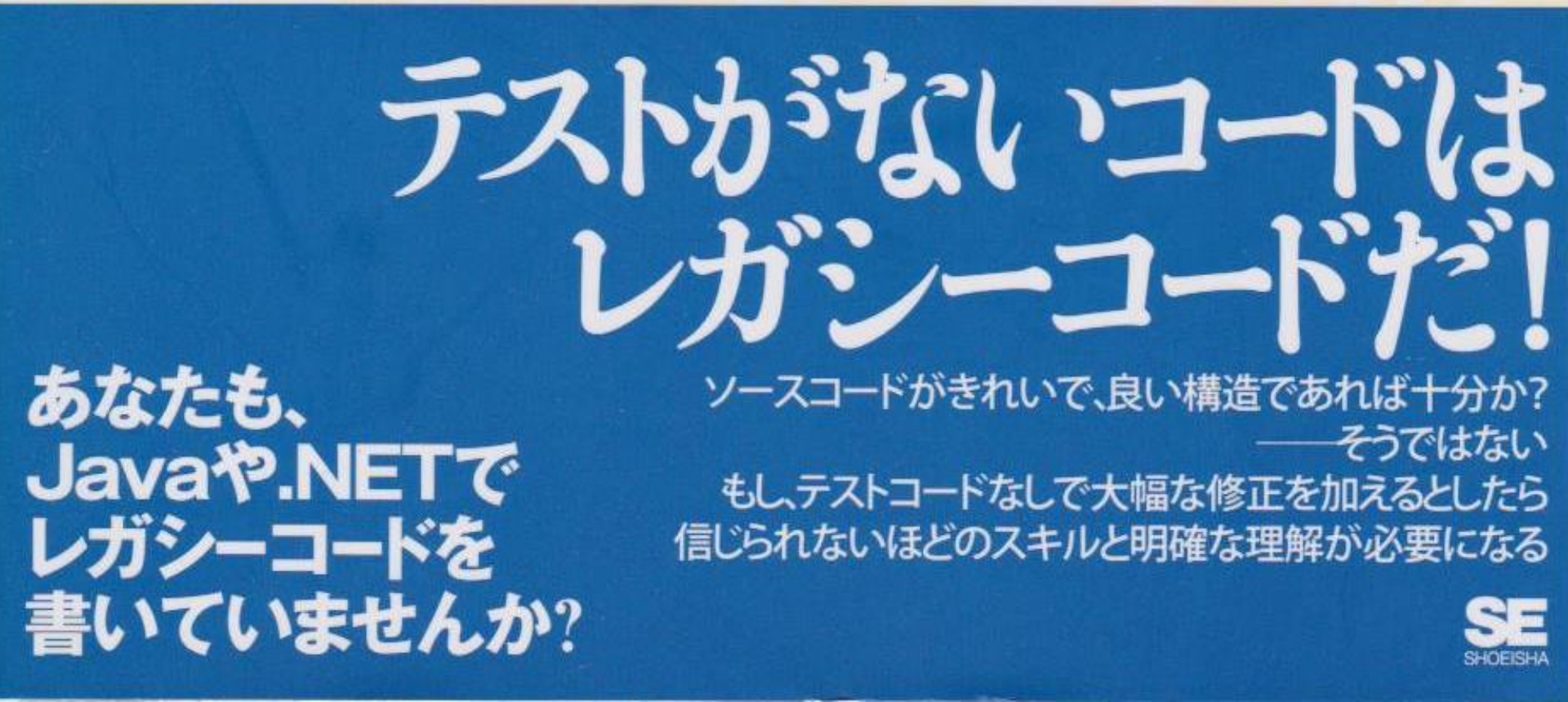
Copyright 2020 Hiroyuki Onaka

パブリッククラウドの普及によるユニットテストのカバー範囲の低下

- Web三層アーキテクチャーからマネジードサービスを活用したアーキテクチャーへの変化・
- アーキテクチャーの分散化それに応じたテスト内でのモックオブジェクトの使用頻度の増加
- 結果としてのユニットテストの旨味の低下

Copyright 2020 Hiroyuki Onaka

「テストがないコードはレガシーコードだ！」



「テストがないコードはレガシーコードだ！」

ご存じ「レガシーコード改善ガイド」の帯の文句

Copyright 2020 Hiroyuki Onaka

「レガシーコード改善ガイド」と私

- 2009年のこと
- 「レガシーコード改善ガイド」の原著「Working Effectively with Legacy Code」の読書会メーリングリストに、訳書のレビューと、タイトルのアイデア出しの依頼のメールが来る

Copyright 2020 Hiroyuki Onaka

https://groups.google.com/forum/#!forum/devtesting-ja

Googleグループの過去ログから

自分が思いついたのは
「レガシーコード改善ガイド」
です。

オビは皆にインパクトを与えた
"To me,legacy code is simply code without tests"
をアレンジするのがよいのではないでしょうか。

Copyright 2020 Hiroyuki Onaka

「テストがないコードはレガシーコードだ」はそういう文脈で発せられたのか

この業界で「レガシーコード」と言うと、多くの場合、理解できず、変更の難しいコードを指す俗語として扱われます。しかし、何年もの間さまざまなチームで深刻なコードの問題を克服する手伝いをしてきて、私は異なる定義にたどり着きました。

私にとって、レガシーコードとは、単にテストのないコードです。

この定義については疑問もありました。コードの良し悪しとテストに何の関係があるのでしょう。私にとってその答えは簡単です。そしてこれが、本書を通じて詳しく述べていきたい核心です。

Copyright 2020 Hiroyuki Onaka

ただ、「レガシーコード改善ガイド」はこういう本である

「3つ目の特徴は、レガシーコードを扱う開発者に対する限りない愛情を感じさせることです。おそらくFeathers氏はコンサルティングの現場において、目の前でレガシーコードでテストを保護して見せながら、技術者たちを励ましているのでしょう。」（訳者あと書きから）

Copyright 2020 Hiroyuki Onaka

レガシーコードに立ち向かうプログラマーに自動テストという武器を授けようとしているのが「レガシーコード改善ガイド」で、「テストがないコードは悪いコード」とは言ってな…

Copyright 2020 Hiroyuki Onaka

言ってるんですけどね

テストのないコードは悪いコードである。どれだけうまく書かれているかは関係ない。どれだけ美しいか、オブジェクト指向か、きちんとカプセル化されているかは関係ない。テストがあれば、検証しながらコードの動きを素早く変更することができる。テストがなければ、コードが良くなっているのか悪くなっているのかが本当にはわからない。

Copyright 2020 Hiroyuki Onaka

自動テストの恩恵

- 品質保証においてプログラマーがカバーできる範囲の飛躍的な向上
- 細かい単位でプログラムに対するフィードバックを得ることによるプログラムの構造の改善
- 継続的インテグレーションなど、デリバリーのプロセスのシステム化

Copyright 2020 Hiroyuki Onaka

自動テストに対する評価

「ソフトウェア開発の分野において、かくも少ない行数のコードが、かくも多数のコードを守ったことはない」

テスティングフレームワークJUnitに対するマーチン・ファウラーの評

Copyright 2020 Hiroyuki Onaka

もうソフトウェア開発が「ストーリーポイント／自動テストがない世界」に戻ることはない。

ではその時に、私たちはそのようなプラクティスにどのような役割を求めればいいのか？

Copyright 2020 Hiroyuki Onaka

XPとは

Copyright 2020 Hiroyuki Onaka

XPと「信頼」

- 「エクストリームプログラミング」には「信頼」という言葉がよく出てきます。
- grepしたら54回出てきた

Copyright 2020 Hiroyuki Onaka

なぜXPは「信頼」という言葉を繰り返すのか

XPを根付かせるうえで、「信頼」に基底を置きたいから

Copyright 2020 Hiroyuki Onaka

技術力と信頼

「技術力は信頼関係につながる。作業を正確に見積もり、最初から品質の高いものを届け、高速なフィードバックループを構築すれば、あなたは信頼されるパートナーになれる。」

Copyright 2020 Hiroyuki Onaka

チームと信頼

「チームメイトはあなたを信頼し、あなたもまたチームメイトを信頼する。」

(「テスト駆動開発」)から

Copyright 2020 Hiroyuki Onaka

XPチームとは信頼を基盤においたコミュニティー

- XPの中で「価値・原則・プラクティス」の枠組みを実践する中で、プログラマーが社会のの一員として認められること。
- 今日テーマとしてあげた「ストーリーポイント」「自動テスト」についても、その中で力を発揮する使い方を望みます。

Copyright 2020 Hiroyuki Onaka

これからのアジャイル

- アジャイルマニフェストから来年で20年
- 「アジャイルサムライ」日本語版から来年で10年
- アジャイルの考えで育ったマネジメント層がこれから増えてくる
- XPの「ソーシャルチェンジ」を実現できるチーム作りはまだこれから

Copyright 2020 Hiroyuki Onaka

ありがとうございました！

- 大中浩行(Onaka,Hiroyuki)
- @setoazusa
- グロース・アーキテクチャ＆チームス株式会社
- http://hiroyuki.fieldnotes.jp/

Copyright 2020 Hiroyuki Onaka